PERCORSO DI FORMAZIONE • 160 ORE

CLOUD ENGINEER ACADEMY

Fondamenti e competenze operative

Dispensa didattica per i discenti

01	Cloud e Azure	24 ore
02	Amministrazione Azure	32 ore
03	DevOps, Git e container	32 ore
04	Automazione e CI/CD	32 ore
05	Kubernetes e AKS	40 ore

Teoria • Esempi commentati • Laboratori • Progetto finale

20 giornate | Edizione didattica del 9 novembre 2026

Indice

Come utilizzare questa dispensa 4

Mappa del percorso e prerequisiti 5

1.1 • Che cosa significa lavorare nel cloud 6

1.2 • Come è organizzato Microsoft Azure 7

1.3 • Calcolo, rete e archiviazione 8

1.4 • Sicurezza, governance e costi 9

Laboratorio 1 • Il primo ambiente Azure 10

2.1 • Amministrare utenti, gruppi e accessi 11

2.2 • Reti virtuali e accessi controllati 12

2.3 • VM, servizi applicativi e archiviazione 13

2.4 • Azure CLI, PowerShell e monitoraggio 14

Laboratorio 2 • Storage e diagnosi operativa 15

3.1 • DevOps: collaborazione e feedback 16

3.2 • Git: working tree, staging e commit 17

3.3 • Branch, merge e pull request 18

3.4 • Container: immagine, processo e persistenza 19

Laboratorio 3A • Academy Portal in locale 20

Laboratorio 3B • Test locale e registry Azure 21

4.1 • Automazione e Infrastructure as Code 22

4.2 • Bicep: una rete descritta come codice 23

4.3 • Terraform: configurazione, plan e state 24

4.4 • Gestire stato, parametri e credenziali 25

Indice • seguito

4.5 • CI/CD e lettura di una pipeline YAML 26

Laboratorio 4A • Verifiche automatiche in GitHub 27

Laboratorio 4B • Deployment infrastrutturale 28

4.6 • La stessa logica in Azure Pipelines 29

5.1 • Kubernetes: stato desiderato e riconciliazione 30

5.2 • Deployment, risorse e health check 31

5.3 • Service, Ingress e accesso all'applicazione 32

5.4 • ConfigMap, Secret e dati persistenti 33

5.5 • AKS: preparare il laboratorio 34

Laboratorio 5A • Il Deployment del portale 35

Laboratorio 5B • Service e verifica funzionale 36

5.6 • Helm: pacchetti e configurazione 37

5.7 • Troubleshooting Kubernetes e AKS 38

5.8 • Automatizzare il rilascio applicativo 39

Progetto finale • Dal repository al cluster 40

Checklist operativa e chiusura del laboratorio 41

Ripasso commentato • Cloud, Azure e Git 42

Ripasso commentato • Automazione e Kubernetes 43

Glossario • Cloud, sviluppo e automazione 44

Glossario • Kubernetes e operatività 45

Fonti e approfondimenti • Azure e DevOps 46

Fonti e approfondimenti • IaC e Kubernetes 47

Come utilizzare questa dispensa

Questa dispensa sviluppa il programma «Cloud Engineer Academy - Fondamenti e competenze operative», della durata di 160 ore. È pensata come supporto alle lezioni, ai laboratori e allo studio individuale: i concetti sono collegati alle attività che un Cloud Engineer junior deve imparare a svolgere e documentare.

Metodo di studio

Per ogni argomento procedi in quattro passaggi: comprendi lo scopo, osserva un esempio, riproduci l'attività in laboratorio e spiega il risultato. Un comando eseguito senza comprenderne gli effetti non costituisce, da solo, una competenza operativa. Devi saper riconoscere il contesto in cui usarlo e verificare che abbia prodotto il risultato atteso.

Il filo conduttore è **Academy Portal**, una piccola applicazione web statica. Partirai dall'organizzazione delle risorse Azure; passerai al repository e al container; automatizzerai alcune attività; infine distribuirai l'applicazione su Kubernetes. La semplicità dell'applicazione consente di concentrarsi sull'infrastruttura.

Convenzioni e ambiente

Gli esempi Azure CLI usano **Bash**. I blocchi PowerShell sono indicati espressamente. Esegui i comandi dalla directory del progetto, salvo diversa indicazione. Le variabili definite in una sessione non sono automaticamente disponibili in un'altra: se riapri la shell, ridefiniscile.

I valori tra parentesi angolari, come `<ID_SOTTOSCRIZIONE>`, sono segnaposto da sostituire. Nomi, regione, dimensioni e versioni vanno concordati con il docente. Le schermate del portale possono cambiare: cerca il nome del servizio o dell'operazione.

I laboratori cloud richiedono una sottoscrizione autorizzata e possono generare costi. Usa risorse didattiche dedicate, verifica gli importi e rimuovi ciò che non serve. I budget inviano avvisi: non costituiscono automaticamente un blocco della spesa. [F03]

Cosa contiene e cosa verificare

Troverai teoria, esempi commentati, cinque laboratori principali, esercizi di diagnosi, un progetto finale e risposte orientative alle domande di ripasso. Gli esempi sono didattici: non sono stati eseguiti nella sottoscrizione della classe. Prima dell'uso il docente deve verificare permessi, quote, servizi consentiti e versioni installate.

Mappa del percorso e prerequisiti

Modulo del programma	Ore	Risultato operativo
1. Cloud e Microsoft Azure Fundamentals	24	Leggere l'organizzazione di Azure e scegliere servizi di base
2. Amministrazione operativa di Microsoft Azure	32	Configurare risorse, accessi, reti e controlli
3. DevOps, Git e container	32	Collaborare sul codice e produrre un'immagine container
4. Automazione, Infrastructure as Code e CI/CD	32	Descrivere infrastrutture e automatizzare verifiche e rilasci
5. Orchestrazione con Kubernetes e AKS	40	Distribuire, aggiornare e diagnosticare un'applicazione
Totale	160	20 giornate previste dal programma

Prerequisiti da consolidare

Occorre saper gestire file e cartelle, distinguere un programma da un processo, comprendere indirizzo IP e porta, modificare un file di testo e leggere messaggi di errore. Non è richiesta una precedente esperienza professionale sul cloud; serve però familiarità progressiva con il terminale.

HTTP è un protocollo di comunicazione applicativo. Una risposta 200 indica normalmente successo; 403 indica un accesso rifiutato; 404 una risorsa non trovata; 500 un errore del server. **DNS** traduce nomi in indirizzi. Un nome non risolto e una porta bloccata sono problemi diversi, anche se il browser mostra in entrambi i casi una pagina non raggiungibile.

Strumenti e responsabilità

Il docente predispone account, sottoscrizione, budget, autorizzazioni e cluster o sandbox. Il discente usa un editor, Git, Docker, Azure CLI, il modulo Az di PowerShell, Terraform, kubectl e Helm secondo il laboratorio. Cloud Shell è adatta a molte attività Azure, ma non va considerata un ambiente Docker locale equivalente.

Annota nel README le versioni ottenute con `git --version`, `docker version`, `az version`, `terraform version`, `kubectl version --client` e `helm version`. Questo permette di confrontare risultati ottenuti su postazioni diverse.

Evidenze da conservare

Per ogni laboratorio registra obiettivo, prerequisiti, comandi significativi, esito, errore incontrato e soluzione. Conserva configurazioni e documentazione nel repository; escludi password, chiavi private, token, file di stato e dati personali. Uno screenshot aiuta a documentare, ma non sostituisce i file necessari a ricreare il risultato.

1.1 • Che cosa significa lavorare nel cloud

Il cloud computing rende disponibili capacità di calcolo, archiviazione e rete attraverso servizi attivabili su richiesta. L'utente descrive ciò di cui ha bisogno e accede alle risorse attraverso API, portali e strumenti. Questo riduce la necessità di acquistare e installare hardware per ogni nuovo ambiente, ma richiede disciplina nella gestione di consumi e configurazioni.

Scalabilità significa poter aumentare o ridurre la capacità. La scalabilità verticale modifica le risorse di un'istanza, per esempio memoria e CPU; quella orizzontale aggiunge o rimuove istanze. **Elasticità** è l'adattamento della capacità al carico, spesso tramite automazione. Aumentare le repliche non risolve automaticamente un collo di bottiglia nel database.

Modelli di servizio e responsabilità

Modello	Cosa ricevi	Esempio e attività del team
IaaS	Infrastruttura virtuale	VM: gestire sistema operativo, aggiornamenti e applicazione
PaaS	Piattaforma gestita	App Service: distribuire codice e configurare l'applicazione
SaaS	Applicazione utilizzabile	Servizio software: gestire utenti, accessi e uso dei dati

Il fornitore gestisce porzioni diverse dello stack a seconda del servizio. I dati e le scelte di accesso restano una responsabilità dell'organizzazione: un servizio gestito non elimina la necessità di configurarlo correttamente. [F01]

Modelli di distribuzione

Nel cloud pubblico le risorse sono offerte da un provider a più clienti con separazione logica. Un cloud privato è dedicato a un'organizzazione. Un ambiente ibrido collega sistemi interni e cloud; un approccio multicloud impiega più provider. «Ibrido» e «multicloud» descrivono aspetti differenti e possono coesistere.

Esempio ragionato

Un piccolo portale informativo non richiede necessariamente una VM. Se basta pubblicare un'applicazione web, una piattaforma gestita può ridurre le attività sistemistiche. Se un software impone un particolare sistema operativo o componenti installati a livello macchina, una VM può essere più adatta. La scelta parte dai requisiti, non dal servizio più conosciuto.

Da ricordare: velocità di attivazione non significa assenza di pianificazione. Prima di creare una risorsa chiarisci chi la usa, quali dati tratta, quanto deve restare attiva e come sarà rimossa.

1.2 • Come è organizzato Microsoft Azure

Azure separa il piano dell'identità dall'organizzazione delle risorse. Il tenant Microsoft Entra ID rappresenta il contesto di identità; una sottoscrizione costituisce un ambito per risorse, quote, autorizzazioni e fatturazione. Un account può avere accesso a più tenant o sottoscrizioni: verificare il contesto è il primo controllo operativo.

Gerarchia delle risorse

I management group consentono di organizzare più sottoscrizioni e applicare regole a livelli superiori. Ogni sottoscrizione contiene resource group; questi raggruppano risorse correlate per gestione e ciclo di vita. Una VM, una rete virtuale e un account di archiviazione sono esempi di risorse.

Un resource group è un contenitore logico, non una rete e non una cartella del filesystem. La sua eliminazione elimina le risorse contenute, salvo impedimenti come blocchi o dipendenze. Per il laboratorio è utile un gruppo dedicato al singolo discente, così da rendere identificabili consumi e operazioni di pulizia.

Regioni, zone e disponibilità

Una regione è un'area geografica nella quale Azure offre servizi. Le zone di disponibilità separano infrastrutture all'interno di una regione supportata. La scelta della regione dipende da latenza, disponibilità del servizio, vincoli sui dati e costi. Non tutti i servizi o gli SKU sono disponibili ovunque.

La presenza di più zone non rende automaticamente un'applicazione resiliente: occorre verificare come sono distribuiti i componenti e come viene gestito il guasto di una dipendenza. Due istanze nella stessa macchina non proteggono dal guasto di quella macchina.

Nomi e tag

Adotta nomi leggibili, per esempio `rg-academy-al01`, `vnet-academy` e `vm-lab`. I vincoli variano per servizio: gli account storage richiedono nomi globalmente univoci con regole specifiche. Non inserire informazioni riservate nei nomi.

I tag sono coppie chiave-valore, come `project=academy`, `environment=lab` e `owner=al01`. Servono a classificare e analizzare le risorse; non attribuiscono permessi e non garantiscono, da soli, che tutti i costi siano correttamente ripartiti. Usa uno schema comune per evitare varianti incoerenti.

Esercizio: disegna una sottoscrizione con due ambienti, sviluppo e test. Indica i resource group, i proprietari operativi e le risorse che potrebbero avere un ciclo di vita separato.

1.3 • Calcolo, rete e archiviazione

Scegliere il servizio di calcolo

Una **Virtual Machine** offre controllo sul sistema operativo. È utile quando occorrono configurazioni specifiche, ma implica manutenzione, patch e gestione della macchina. **App Service** ospita applicazioni web su una piattaforma gestita. **Azure Container Instances** consente di eseguire container senza amministrare un cluster. **AKS** aggiunge l'orchestrazione Kubernetes e comporta maggiori attività progettuali e operative.

Una semplice pagina web non richiede necessariamente Kubernetes. Nel corso useremo AKS perché l'obiettivo didattico è imparare l'orchestrazione: nella realtà la complessità deve essere giustificata da requisiti e competenze del team.

Scegliere la persistenza

Tipo	Uso tipico	Domanda progettuale
Blob Storage	Oggetti e file, come immagini e documenti	Chi deve leggere e scrivere gli oggetti?
Azure Files	Condivisioni file	L'applicazione richiede accesso tramite filesystem condiviso?
Managed Disks	Dischi delle VM	Quali prestazioni e capacità servono alla macchina?
Database gestito	Dati strutturati e interrogazioni	Quali relazioni, transazioni e query sono richieste?

Un oggetto Blob non si comporta come una tabella relazionale. La persistenza va scelta in base al modello di accesso, non soltanto al volume dei dati. Replicazione e ridondanza proteggono da determinati guasti, ma non sostituiscono automaticamente versionamento e backup contro cancellazioni o modifiche errate. [F05]

La rete come percorso della richiesta

Quando un browser contatta un'applicazione, deve risolverne il nome, raggiungere un indirizzo, aprire una connessione alla porta corretta e ottenere una risposta. Firewall, regole di rete, autenticazione e applicazione intervengono in momenti diversi. Un errore va localizzato lungo questo percorso.

La VNet è la rete privata di Azure; le subnet ne suddividono lo spazio di indirizzamento. Un indirizzo pubblico rende possibile un punto di accesso da Internet, ma la raggiungibilità dipende anche da routing, filtri e servizio in ascolto. [F04]

Caso pratico: devi pubblicare un sito che mostra documenti scaricabili. Separa il servizio web dall'archivio dei file; definisci chi carica i documenti, se sono pubblici o riservati e come recuperarli dopo una cancellazione accidentale.

1.4 • Sicurezza, governance e costi

Identità prima delle autorizzazioni

L'autenticazione verifica chi sei; l'autorizzazione stabilisce cosa puoi fare. Microsoft Entra ID gestisce identità e autenticazione, mentre Azure RBAC attribuisce permessi sulle risorse. Un'assegnazione RBAC combina identità, ruolo e ambito. Il ruolo Reader permette di osservare molte configurazioni, mentre Contributor consente la gestione delle risorse ma non attribuisce generalmente il potere di assegnare ruoli. [F02]

Il principio del minimo privilegio richiede il ruolo sufficiente sullo scope più ristretto utile. Concedere Owner sull'intera sottoscrizione per risolvere un singolo errore è una scorciatoia che impedisce di capire il problema.

Governance operativa

Azure Policy aiuta a valutare o imporre regole sulle configurazioni, per esempio regioni ammesse o requisiti sui tag. RBAC stabilisce chi può agire; Policy definisce condizioni che le risorse devono rispettare. Un utente autorizzato può quindi ricevere un rifiuto dovuto a una policy.

I lock possono ostacolare modifiche o eliminazioni accidentali sul piano di gestione. Non sono backup e non sostituiscono un controllo degli accessi. In un laboratorio vanno compresi prima di applicarli, perché possono impedire la pulizia finale.

Leggere il costo totale

Il costo di una soluzione comprende più elementi: capacità di calcolo, dischi, archiviazione, operazioni, trasferimento dati, logging e servizi di rete. Una VM deallocata può lasciare costi per dischi e altre risorse. Anche un cluster con pochi workload può mantenere nodi attivi.

Imposta un budget sullo scope del laboratorio e consulta Cost Analysis. Gli avvisi di budget non sono in tempo reale e non arrestano da soli i servizi. Per una stima usa il calcolatore ufficiale selezionando regione, SKU e ore di utilizzo; confronta poi la stima con i consumi effettivi. [F03]

Ripasso del modulo 1

1. Quando sceglieresti PaaS invece di IaaS per un portale web?

2. Perché un resource group non può essere considerato una subnet?

3. Un budget di 20 euro garantisce che non si possano spendere 21 euro? Motiva.

4. Perché ridondanza e backup rispondono a esigenze differenti?

Le risposte orientative si trovano nella sezione finale. Prima di leggerle, formula una spiegazione utilizzando un esempio.

Laboratorio 1 • Il primo ambiente Azure

Obiettivo: identificare il contesto corretto e creare un resource group dedicato. **Prerequisiti:** account autorizzato, Azure CLI o Cloud Shell Bash, permesso di creare il gruppo. **Evidenza:** breve scheda dell'ambiente con nomi, regione e tag, senza credenziali.

1. Controllare identità e sottoscrizione

Da una postazione locale esegui il login. In Cloud Shell la sessione potrebbe essere già autenticata: verifica comunque il contesto.

```
az login
az account list --output table
az account set --subscription '<ID_SOTTOSCRIZIONE>'
az account show --query '{name:name,id:id}' -o table
```

Non proseguire se il nome della sottoscrizione non corrisponde a quello assegnato. La disponibilità di un account non autorizza a usare qualunque ambiente aziendale.

2. Definire variabili e creare il gruppo

Sostituisci `al01` con il codice concordato. La regione è un esempio: usa quella consentita dal laboratorio.

```
LAB_RG='rg-academy-al01'
LAB_LOCATION='westeurope'
az group create --name "$LAB_RG" \
  --location "$LAB_LOCATION" \
  --tags project=academy environment=lab owner=al01
az group show --name "$LAB_RG" \
  --query '{name:name,location:location,tags:tags}' -o json
```

3. Verificare dal portale e documentare

Apri il gruppo dal portale e confronta nome, regione e tag con l'output della CLI. Osserva che portale e terminale operano sulle stesse risorse. Crea una scheda `docs/lab01.md` con scopo del gruppo e criteri di eliminazione; sarà inserita nel repository nel modulo 3.

Esito atteso: gruppo presente, tag corretti e sottoscrizione verificata. Se ricevi `AuthorizationFailed`, controlla il ruolo assegnato e lo scope; se ricevi un errore di policy, leggi il dettaglio della regola. Non cambiare sottoscrizione per aggirare il vincolo.

Chiusura: conserva il gruppo se sarà usato nei moduli successivi. Configura con il docente il monitoraggio del costo. Eliminazione e verifica delle risorse residue sono trattate nell'ultima parte della dispensa.

2.1 • Amministrare utenti, gruppi e accessi

L'amministrazione parte dalla richiesta: quale persona o applicazione deve svolgere quale attività e su quali risorse? Il processo deve produrre un'assegnazione comprensibile e verificabile. Un elenco di utenti con privilegi ampi è difficile da mantenere e rende più complessa la revoca.

Utenti, gruppi e identità applicative

Un utente rappresenta una persona. Un gruppo raccoglie identità cui assegnare permessi comuni. Un service principal rappresenta un'applicazione nel tenant. Una managed identity permette a una risorsa supportata di ottenere un'identità senza dover gestire direttamente una password applicativa.

I ruoli Microsoft Entra per l'amministrazione della directory e i ruoli Azure sulle risorse hanno finalità differenti. Non presumere che poter creare una VM consenta anche di creare utenti, o che un ruolo nella directory dia automaticamente accesso ai dati archiviati.

Il controllo degli accessi ai dati

Distingui **management plane**, usato per creare e configurare servizi, e **data plane**, usato per leggere o scrivere i contenuti. Un permesso di gestione su uno storage account non equivale necessariamente a un ruolo esplicito di lettura Blob tramite Microsoft Entra. Per i laboratori assegna i ruoli dati necessari con il docente e attendi l'eventuale propagazione.

Procedura di assegnazione

1. Identifica il destinatario tramite identificatore corretto, evitando omonimie.
2. Definisci lo scope minimo: risorsa, gruppo o sottoscrizione.
3. Seleziona il ruolo che copre le operazioni richieste.
4. Verifica che chi assegna il ruolo abbia il permesso di farlo.
5. Prova sia l'operazione consentita sia una che deve restare vietata.
6. Documenta finalità e data di revisione o revoca dell'accesso.

Esempio: un collega deve leggere i file di un contenitore Blob senza riconfigurare lo storage. Il requisito riguarda i dati, non la gestione del servizio. La soluzione va cercata nei ruoli dati, sullo scope appropriato, e non in un generico privilegio elevato. [F02]

Errore frequente: aggiungere nuovi ruoli a ogni tentativo senza rimuovere quelli inutili. Prima di cambiare autorizzazioni, verifica account attivo, scope e operazione negata.

2.2 • Reti virtuali e accessi controllati

Una rete virtuale utilizza uno spazio di indirizzi, per esempio `10.20.0.0/16`. Una subnet può usare una parte di quello spazio, per esempio `10.20.1.0/24`. La notazione CIDR indica quanti bit identificano la rete: /24 contiene 256 indirizzi complessivi, ma Azure ne riserva alcuni, quindi non sono tutti assegnabili.

Pianificare gli indirizzi serve a evitare sovrapposizioni quando reti diverse vengono collegate. Due VNet con lo stesso intervallo possono creare difficoltà di integrazione. Il nome della subnet non stabilisce le regole di accesso: è la configurazione effettiva a determinarle.

Network Security Group

Un NSG filtra traffico in ingresso e in uscita in base a sorgente, destinazione, porta, protocollo e priorità. Le regole si valutano secondo priorità; i numeri più bassi precedono quelli più alti. Occorre leggere anche le regole predefinite. Un NSG non è un firewall applicativo e non verifica la correttezza funzionale di una pagina web. [F04]

Creare una rete didattica

Le variabili `LAB_RG` e `LAB_LOCATION` provengono dal laboratorio 1. I comandi creano una rete senza pubblicare un servizio su Internet.

```
az network vnet create --resource-group "$LAB_RG" \
  --name vnet-academy --location "$LAB_LOCATION" \
  --address-prefixes 10.20.0.0/16 \
  --subnet-name snet-app --subnet-prefixes 10.20.1.0/24
az network nsg create --resource-group "$LAB_RG" \
  --name nsg-academy --location "$LAB_LOCATION"
az network vnet subnet update --resource-group "$LAB_RG" \
  --vnet-name vnet-academy --name snet-app \
  --network-security-group nsg-academy
```

Diagnosi per livelli

Per un accesso fallito verifica: nome DNS, indirizzo atteso, routing, regole NSG, eventuale firewall del sistema operativo, processo in ascolto e autenticazione applicativa. Se il servizio ascolta soltanto su `127.0.0.1`, una regola di rete permissiva non lo renderà raggiungibile da altre macchine.

Esercizio: descrivi come consentire SSH soltanto dal percorso amministrativo autorizzato. Spiega perché aprire la porta 22 verso qualsiasi origine non è una buona configurazione predefinita. Nel laboratorio usa il percorso privato predisposto dal docente, senza inventare accessi pubblici.

2.3 • VM, servizi applicativi e archiviazione

Gestire una macchina virtuale

Prima di creare una VM definisci immagine del sistema operativo, dimensione, dischi, rete, autenticazione e modalità di accesso. La dimensione deve essere disponibile nella regione e compatibile con le quote. Per Linux, l'uso di chiavi SSH evita di basare l'accesso su una password riutilizzata.

L'esempio seguente richiede la rete della pagina precedente, una chiave pubblica presente e un percorso amministrativo privato predisposto. Non crea un indirizzo pubblico. La disponibilità dello SKU va verificata dal docente.

```
az vm create --resource-group "$LAB_RG" --name vm-lab \
  --image Ubuntu2204 --size Standard_B2s \
  --admin-username azureuser \
  --ssh-key-values "$HOME/.ssh/id_ed25519.pub" \
  --vnet-name vnet-academy --subnet snet-app \
  --public-ip-address "" --nsg ""
az vm get-instance-view -g "$LAB_RG" -n vm-lab \
  --query 'instanceView.statuses[].displayStatus' -o table
```

L'assenza di IP pubblico è intenzionale. Per accedere servono connettività privata e regole coerenti, ad esempio tramite il laboratorio predisposto. Non è un errore da correggere aggiungendo automaticamente un IP pubblico.

Stato della VM e costo

Una VM arrestata dal sistema operativo può restare allocata. La deallocazione rilascia la capacità di calcolo, mentre dischi e altre componenti possono continuare a generare costi. Usa `az vm deallocate -g "$LAB_RG" -n vm-lab` quando previsto; prima di farlo verifica che nessun lavoro debba essere salvato.

Applicazioni gestite

In App Service distingue l'applicazione dal piano che fornisce capacità. La configurazione include runtime, impostazioni, identità, rete e logging. Un deployment riuscito indica che i file sono stati distribuiti, non che l'applicazione risponda correttamente: esegui sempre una verifica HTTP e controlla i log.

Per Blob Storage, una configurazione prudente evita accesso pubblico ai contenitori e usa autenticazione Microsoft Entra quando applicabile. Per dati importanti definisci conservazione, versionamento e ripristino. Esegui almeno una prova di recupero: sapere che un'opzione è abilitata non equivale a saperla utilizzare.

2.4 • Azure CLI, PowerShell e monitoraggio

Portale, CLI e PowerShell sono interfacce differenti verso gli stessi servizi. Il portale aiuta l'esplorazione; gli strumenti da terminale rendono ripetibili operazioni e controlli. Non trasferire automaticamente sintassi Bash in PowerShell: variabili, continuazioni di riga e gestione degli oggetti differiscono.

Esempio equivalente in PowerShell

Con il modulo Az installato, seleziona esplicitamente la sottoscrizione. Esegui questo blocco in PowerShell, non nella shell Bash usata per gli altri esempi.

```
Connect-AzAccount
Set-AzContext -Subscription '<ID_SOTTOSCRIZIONE>'
$LabRg = 'rg-academy-al01'
Get-AzResourceGroup -Name $LabRg
Get-AzResource -ResourceGroupName $LabRg |
  Select-Object Name, ResourceType, Location
```

In PowerShell la pipeline trasporta normalmente oggetti. `Select-Object` seleziona proprietà, non colonne testuali. In Azure CLI `--query` filtra il risultato con espressioni JMESPath, mentre `--output` ne controlla la rappresentazione. Per automatizzare evita di basarti sulla posizione di una colonna formattata per lettura umana.

Che cosa osservare

Le **metriche** sono misure nel tempo, come CPU o numero di richieste. I **log** descrivono eventi con dettagli utili alla diagnosi. Le **tracce** seguono l'esecuzione di richieste tra componenti. Azure Monitor raccoglie e analizza telemetria, ma occorre configurare le sorgenti necessarie e valutarne i costi. [F06]

L'Activity Log aiuta a ricostruire operazioni sul piano di gestione. Non va confuso con i log applicativi. Una VM creata con successo può ospitare un'applicazione che non si avvia: occorre consultare il livello giusto.

Un allarme utile

Un allarme deve avere condizione, finestra di osservazione, destinatario e azione prevista. «CPU superiore all'80%» non basta: definisci per quanto tempo e su quale risorsa. Verifica inoltre se una CPU alta indica davvero un problema per gli utenti.

Ripasso: quale differenza c'è tra errore di autenticazione e autorizzazione? Quali costi possono restare dopo la deallocazione? Perché un codice HTTP 500 orienta la diagnosi in modo diverso da un timeout di connessione?

Laboratorio 2 • Storage e diagnosi operativa

Obiettivo: creare uno storage account e verificare l'accesso ai dati con identità. **Prerequisiti:** laboratorio 1, nome globalmente univoco e ruolo dati adeguato. Il docente assegna `Storage Blob Data Contributor` sullo scope del laboratorio a chi deve caricare file.

1. Creare account e contenitore

Sostituisci il nome con 3-24 caratteri alfanumerici minuscoli, globalmente univoci. Il contenitore si chiama `materiali`.

```
LAB_STORAGE='<NOME_STORAGE_UNIVOCO>'
az storage account create -g "$LAB_RG" -n "$LAB_STORAGE" \
  -l "$LAB_LOCATION" --sku Standard_LRS --kind StorageV2 \
  --allow-blob-public-access false --min-tls-version TLS1_2
az storage container create --name materiali \
  --account-name "$LAB_STORAGE" --auth-mode login
```

2. Caricare e verificare un file

Crea localmente `verifica.txt` con una breve frase, senza dati personali. Esegui poi:

```
az storage blob upload --account-name "$LAB_STORAGE" \
  --container-name materiali --name verifica.txt \
  --file verifica.txt --auth-mode login --overwrite
az storage blob list --account-name "$LAB_STORAGE" \
  --container-name materiali --auth-mode login \
  --query '[].{nome:name,byte:properties.contentLength}' -o table
```

Esito atteso: il file compare con dimensione coerente. Un URL Blob non deve diventare pubblico solo perché il file è stato caricato. Se l'accesso fallisce, controlla identità, ruolo dati, propagazione, rete e nome dell'account; non abilitare l'accesso anonimo per aggirare il problema.

3. Esercizio di diagnosi e consegna

Confronta un nome di contenitore inesistente con un accesso privo del ruolo necessario. Annota il messaggio, l'ipotesi e il controllo svolto. Non revocare ruoli a utenti reali: l'esperimento va organizzato con un account di prova autorizzato.

Consegna `docs/lab02.md` con almeno due errori spiegati, la verifica finale e il costo osservato o l'indicazione che il dato non è ancora disponibile. Mantieni il gruppo per le attività successive, se previsto.

3.1 • DevOps: collaborazione e feedback

DevOps è un insieme di pratiche che avvicina sviluppo e gestione operativa. L'obiettivo è rendere le modifiche piccole, verificabili e distribuibili con continuità. Non coincide con un singolo prodotto e non si esaurisce nella scrittura di pipeline.

Un team efficace rende espliciti i requisiti, registra le modifiche, automatizza verifiche ripetitive e osserva il comportamento dell'applicazione dopo il rilascio. Il feedback operativo torna allo sviluppo: un errore ricorrente deve generare una correzione del sistema o del processo, non soltanto un intervento manuale ripetuto.

Il ciclo di una modifica

Una richiesta diventa un'attività con criteri di accettazione. Lo sviluppatore lavora su un branch, produce un commit e apre una pull request. Le verifiche automatiche e la revisione preparano l'integrazione. La distribuzione porta una versione identificabile nell'ambiente; il monitoraggio verifica il risultato.

Per Academy Portal, una richiesta ben definita può essere: «La home deve mostrare il nome del corso e una pagina health deve rispondere». «Migliorare il sito» è troppo vaga per essere verificata.

Definition of done

Una modifica è completata quando soddisfa requisiti condivisi. Per il progetto didattico: contenuto corretto, test superati, immagine costruibile, documentazione aggiornata, nessun segreto inserito, verifica della versione distribuita e procedura di ripristino descritta.

GitHub e Azure DevOps

GitHub e Azure DevOps offrono repository, revisione e automazione. GitHub Actions esegue workflow; Azure Pipelines organizza pipeline. Una pull request è un processo di proposta e revisione ospitato dalla piattaforma: non è un comando fondamentale di Git.

Il corso usa GitHub Actions per l'esempio completo; una pagina successiva mostra la corrispondenza con Azure Pipelines. La scelta del servizio non cambia la necessità di gestire permessi, branch protetti e credenziali.

Esercizio: descrivi il percorso di una modifica urgente. Indica chi la rivede, quali controlli non possono essere saltati, quale versione viene rilasciata e come verificare il risultato. Una procedura urgente dovrebbe essere più breve, ma ancora tracciabile.

3.2 • Git: working tree, staging e commit

Git registra la storia del progetto attraverso commit. Il working tree contiene i file su cui lavori; l'area di staging seleziona ciò che entrerà nel prossimo commit; il repository conserva la storia. Salvare un file nell'editor non significa averlo registrato in Git. [F07]

Un commit dovrebbe rappresentare una modifica coerente: per esempio aggiungere la pagina iniziale e il relativo controllo. Un messaggio come «Aggiornamento» è meno utile di «Aggiunge titolo e pagina health al portale».

Inizializzare il progetto

Crea una cartella vuota `academy-portal`, entraci e configura nome ed email secondo le indicazioni del corso. I dati saranno associati ai commit.

```
mkdir academy-portal
cd academy-portal
git init -b main
git config user.name 'Nome Cognome'
git config user.email 'email-didattica@example.com'
mkdir -p app docs infra k8s .github/workflows
```

Crea `README.md` con titolo e obiettivo. Crea `.gitignore` includendo almeno le voci sotto. Il file di lock Terraform, invece, va normalmente versionato per conservare la selezione dei provider.

```
.env
*.pem
*.key
.terraform/
*.tfstate
*.tfstate.*
*.tfplan
terraform.tfvars
```

```
git status
git add README.md .gitignore
git diff --staged
git commit -m 'Imposta struttura e regole del repository'
git log --oneline
```

Controllare prima di registrare

`git diff` mostra modifiche non ancora in staging; `git diff --staged` mostra quelle pronte per il commit. L'esclusione in `.gitignore` non elimina dalla storia file già tracciati. Se una credenziale entra nel repository, rimuoverla dal file non basta: occorre revocarla o ruotarla e gestire la storia con il responsabile.

Evidenza: inserisci le schede dei primi laboratori in `docs/`, controlla che non contengano segreti e crea un commit separato per la documentazione.

3.3 • Branch, merge e pull request

Un branch permette di sviluppare una modifica mantenendo distinta la linea principale. Non è una copia indipendente dell'intero progetto: è un riferimento nella storia dei commit. Usa nomi descrittivi, come `feature/home`, e integra modifiche piccole con frequenza.

Flusso collaborativo

Crea sulla piattaforma un repository vuoto, senza README generato, e copia il suo URL. Configura l'autenticazione Git secondo le regole del corso, senza inserire token nell'URL o negli script.

```
git remote add origin '<URL_REPOSITORY>'
git push -u origin main
git switch -c feature/home
# Modifica i file e controlla le differenze.
git add app/index.html
git commit -m 'Aggiunge la home di Academy Portal'
git push -u origin feature/home
```

Apri una pull request dalla piattaforma. Scrivi problema, modifica e verifica eseguita. Il revisore deve poter capire ciò che cambia senza ricostruire tutta la conversazione. Dopo il merge, riallinea il branch locale con `git switch main` e `git pull --ff-only`.

Gestire un conflitto

Un conflitto emerge quando Git non riesce a combinare automaticamente modifiche. Leggi entrambe le intenzioni, modifica il file scegliendo una versione coerente e rimuovi i marcatori di conflitto. Esegui i controlli, aggiungi il file allo staging e completa il merge. Non scegliere una parte soltanto perché compare per prima.

Prima di iniziare un merge controlla `git status`. Se l'integrazione è sbagliata e deve essere annullata mentre è ancora in corso, valuta `git merge --abort`. Per annullare un commit già condiviso, `git revert <COMMIT>` crea una nuova modifica inversa: preserva la storia condivisa, ma va comunque verificato.

Piccolo caso di lavoro

Due colleghi cambiano il titolo della home. Uno usa «Cloud Academy», l'altro «Academy Portal». La soluzione non è tecnica soltanto: serve verificare il requisito condiviso. Il merge deve incorporare il contenuto approvato, mantenendo gli altri cambiamenti utili.

Ripasso: perché un commit non coincide con un push? Cosa deve contenere una pull request utile? In che modo un segreto già registrato resta un problema anche dopo aver aggiornato `.gitignore`?

3.4 • Container: immagine, processo e persistenza

Un'immagine container è un pacchetto contenente applicazione e componenti necessari all'esecuzione. Un container è un'istanza in esecuzione dell'immagine. Più container possono partire dalla stessa immagine e avere cicli di vita distinti.

I container condividono il kernel dell'ambiente host, direttamente o attraverso la macchina virtuale usata dal runtime. Non sono VM complete. La portabilità richiede compatibilità di sistema e architettura: un'immagine costruita per ARM64 non è automaticamente eseguibile su nodi AMD64 senza un'immagine multiarchitettura o supporto adeguato.

Costruire e distribuire

Il Dockerfile descrive come costruire l'immagine. Il registry conserva immagini versionate. Il tag è un'etichetta e può essere riassegnato; il digest identifica il contenuto. Per rendere tracciabile un rilascio usa tag univoci, ad esempio lo SHA del commit, e registra il digest prodotto. [F08]

Una modifica eseguita manualmente dentro un container non aggiorna il Dockerfile. Al successivo avvio potresti perdere la correzione. Modifica il codice o la configurazione sorgente, costruisci una nuova immagine e ridistribuisci.

Porte e dati

`EXPOSE` documenta la porta dell'applicazione ma non pubblica una porta sull'host. L'opzione `-p` di Docker definisce la mappatura. Per il laboratorio, `127.0.0.1:8080:8080` rende il servizio accessibile soltanto dalla postazione locale.

Il filesystem scrivibile del container non è un archivio durevole. Per dati persistenti servono volumi o servizi esterni. La nostra applicazione è statica: non salva dati utente e può essere ricreata senza recuperare lo stato di un container precedente.

Build pulita

Un `.dockerignore` limita ciò che entra nel contesto di build; riduce trasferimenti inutili e possibilità di includere file riservati. Usa immagini di provenienza verificata, controlla gli aggiornamenti e scegli componenti necessari. Un'immagine più piccola non è automaticamente sicura, ma è più semplice da comprendere. [F08]

Esercizio: spiega cosa accade se modifichi `index.html` nella cartella locale dopo una build che lo ha copiato nell'immagine. Perché il container già avviato può continuare a mostrare il contenuto precedente?

Laboratorio 3A • Academy Portal in locale

Obiettivo: costruire e verificare un container web. **Prerequisiti:** repository del modulo 3, Docker attivo e porta locale 8080 disponibile. Lavora su un branch dedicato.

1. Creare i file dell'applicazione

Salva questo contenuto in `app/index.html` e una singola riga `ok` in `app/health.html`.

```
<!doctype html>
<html lang="it">
<head><meta charset="utf-8"><title>Academy Portal</title></head>
<body>
  <h1>Academy Portal</h1>
  <p>Cloud Engineer Academy - versione 1</p>
</body>
</html>
```

Crea `nginx.conf` nella radice del repository. Usiamo una porta non privilegiata e directory temporanee scrivibili dall'utente del container.

```
pid /tmp/nginx.pid;
events {}
http {
  include /etc/nginx/mime.types;
  access_log /dev/stdout;
  error_log /dev/stderr warn;
  client_body_temp_path /tmp/client_temp;
  proxy_temp_path /tmp/proxy_temp;
  fastcgi_temp_path /tmp/fastcgi_temp;
  uwsgi_temp_path /tmp/uwsgi_temp;
  scgi_temp_path /tmp/scgi_temp;
  server {
    listen 8080;
    root /usr/share/nginx/html;
    location / { try_files $uri $uri/ =404; }
  }
}
```

2. Definire il Dockerfile

Il tag di base seguente è didattico e mutabile: prima del laboratorio verifica disponibilità e aggiornamenti; per riproducibilità registra il digest effettivo.

```
FROM nginx:stable-alpine
COPY nginx.conf /etc/nginx/nginx.conf
COPY app/ /usr/share/nginx/html/
USER nginx
EXPOSE 8080
ENTRYPOINT ["nginx"]
CMD ["-g", "daemon off;"]
```

Laboratorio 3B • Test locale e registry Azure

Crea `.dockerignore` con `.git`, `.env`, `.terraform`, `*.tfstate*` e `*.tfplan`, una voce per riga. Esegui dalla radice del repository:

```
docker build -t academy-portal:v1 .
docker run -d --name academy-local \
  -p 127.0.0.1:8080:8080 academy-portal:v1
curl --fail http://127.0.0.1:8080/
curl --fail http://127.0.0.1:8080/health.html
docker logs academy-local
docker rm -f academy-local
```

Esito atteso: home corretta, risposta `ok` sulla health e nessun errore di avvio. La pagina health controlla il web server; non misura dipendenze che questa applicazione non possiede. Se la porta è occupata, scegli un'altra porta host e aggiorna gli URL; non cambiare casualmente la porta interna.

Pubblicare in Azure Container Registry

Per il laboratorio usa un registry ACR predisposto dal docente con modalità autorizzativa RBAC e identità abilitata al push. Il nome è globalmente univoco. Il docente può crearlo come segue, dopo aver verificato budget e permessi:

```
LAB_ACR='<NOME_ACR_UNIVOCO>'
az acr create -g "$LAB_RG" -n "$LAB_ACR" --sku Basic
az acr login --name "$LAB_ACR"
LAB_LOGIN_SERVER=$(az acr show -n "$LAB_ACR" \
  --query loginServer -o tsv)
```

Dopo aver registrato e committato i file, costruisci per l'architettura AMD64 usata nel cluster del corso. Questo passaggio richiede Docker Buildx configurato; su macchine ARM può richiedere emulazione.

```
LAB_TAG=$(git rev-parse --short HEAD)
docker buildx build --platform linux/amd64 \
  -t "$LAB_LOGIN_SERVER/academy-portal:$LAB_TAG" \
  --push .
```

Registra repository, tag e digest, senza password. Il nome di login va letto da Azure e non costruito per supposizione. Il login al registry prova l'autenticazione; il push richiede anche autorizzazione. Nei registry con modalità ABAC i ruoli differiscono: adeguarli con il docente. [F09]

Consegna: Dockerfile, configurazione web, pagine, pull request e scheda con prove HTTP. Il container è pubblicato nel registry, ma non è ancora un'applicazione disponibile agli utenti: sarà distribuito su AKS nel modulo 5.

4.1 • Automazione e Infrastructure as Code

Automatizzare significa rendere eseguibile una procedura definita. Prima di scrivere uno script chiarisci input, prerequisiti, risultato atteso e modalità di gestione dell'errore. Se una procedura manuale è ambigua, uno script può soltanto ripeterne più velocemente gli errori.

Con l'Infrastructure as Code, la descrizione dell'infrastruttura è conservata in file versionabili. Il team può revisionare modifiche, confrontare ambienti e ricostruire il contesto delle decisioni. I file sono parte del progetto e richiedono la stessa cura del codice applicativo.

Imperativo e dichiarativo

Un approccio imperativo specifica azioni: «crea una rete, poi una subnet». Uno dichiarativo descrive lo stato desiderato: «deve esistere una rete con questa subnet». Il motore di deployment calcola le operazioni necessarie. La distinzione non elimina la necessità di comprendere dipendenze e conseguenze.

L'idempotenza indica che ripetere un'operazione con gli stessi input non produce cambiamenti aggiuntivi indesiderati. Non significa che qualsiasi script o configurazione IaC sia sempre sicuro da rilanciare. Nomi generati casualmente, dati esterni e operazioni distruttive richiedono attenzione.

Bicep e Terraform

Aspetto	Bicep	Terraform
Ambito	Linguaggio orientato alle risorse Azure	Ecosistema di provider per diversi servizi
Descrizione	Risorse, parametri, moduli e output	Risorse, variabili, moduli e output in HCL
Anteprima	Operazione what-if	Comando plan
Tracciamento	Deployment gestito da Azure Resource Manager	State da conservare e proteggere

Bicep rende più leggibile la definizione delle risorse Azure. Terraform mantiene una relazione tra configurazione e oggetti gestiti tramite lo state. Non affidare la stessa risorsa contemporaneamente a due strumenti senza una strategia esplicita di migrazione. [F10, F12]

Drift e verifica

Il drift è una divergenza tra configurazione attesa e ambiente effettivo, spesso causata da modifiche manuali. Non ogni differenza deve essere annullata: prima comprendi se deriva da una correzione urgente da riportare nel codice. L'obiettivo è avere una fonte affidabile e una procedura condivisa.

4.2 • Bicep: una rete descritta come codice

L'esempio crea una rete distinta da quella del modulo 2. Salvalo in `infra/main.bicep`. Il resource group deve già esistere. I parametri rendono esplicite le parti variabili; gli output restituiscono informazioni utili ad altri passaggi.

```
param location string = resourceGroup().location
param vnetName string = 'vnet-iac'

resource vnet 'Microsoft.Network/virtualNetworks@2023-11-01' = {
  name: vnetName
  location: location
  tags: {
    project: 'academy'
    environment: 'lab'
  }
  properties: {
    addressSpace: {
      addressPrefixes: [ '10.30.0.0/16' ]
    }
    subnets: [
      {
        name: 'snet-app'
        properties: {
          addressPrefix: '10.30.1.0/24'
        }
      }
    ]
  }
}

output vnetId string = vnet.id
```

Validare, osservare, distribuire

```
az bicep build --file infra/main.bicep
az deployment group what-if -g "$LAB_RG" \
  --template-file infra/main.bicep
az deployment group create -g "$LAB_RG" \
  --name academy-network --template-file infra/main.bicep
```

La compilazione verifica la struttura Bicep, ma non prova tutti i requisiti dell'ambiente. Il what-if anticipa i cambiamenti e può avere limiti di analisi: leggi avvisi e risorse non valutate. Dopo la distribuzione verifica la rete effettiva e conserva l'esito. [F11]

Esercizio: cambia un tag e riesegui il what-if. Poi modifica il nome della rete: osserva che il nome identifica una risorsa diversa, non una semplice etichetta. Non applicare il secondo cambiamento senza valutarne l'effetto.

4.3 • Terraform: configurazione, plan e state

Crea una cartella `infra/terraform` e salva il file `main.tf` seguente. L'esempio gestisce un resource group separato, per non sovrapporsi alle risorse dei laboratori precedenti. Usa Azure CLI per autenticarti prima di eseguire Terraform.

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}
provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}
variable "subscription_id" { type = string }
variable "resource_group_name" { type = string }
variable "location" {
  type    = string
  default = "westeurope"
}
resource "azurerm_resource_group" "lab" {
  name     = var.resource_group_name
  location = var.location
  tags     = { project = "academy", environment = "lab" }
}
output "resource_group_id" {
  value = azurerm_resource_group.lab.id
}
```

Il ciclo operativo

Dalla cartella Terraform, passa le variabili tramite l'ambiente. Adegua regione e nome se richiesto dalla classe.

```
export TF_VAR_subscription_id='<ID_SOTTOSCRIZIONE>'
export TF_VAR_resource_group_name='rg-academy-tf-al01'
terraform init
terraform fmt
terraform validate
terraform plan -out=lab.tfplan
terraform apply lab.tfplan
```

`init` prepara provider e backend; `validate` verifica la configurazione; `plan` mostra le operazioni proposte; `apply` le esegue. Il file di lock del provider va conservato nel repository. Non interpretare l'assenza di errori sintattici come prova di permessi o quote sufficienti. [F12]

4.4 • Gestire stato, parametri e credenziali

Lo state Terraform

Lo state collega le risorse dichiarate agli oggetti effettivi. Non è un file da modificare manualmente per «aggiustare» un errore. Per il laboratorio individuale può essere locale; nel lavoro di gruppo serve un backend condiviso, protetto e con gestione del locking, per evitare scritture concorrenti.

Lo state e i file di piano possono contenere dati sensibili. La marcatura `sensitive` limita la visualizzazione di un valore, ma non ne garantisce l'assenza dallo state. Escludi questi file da Git e gestiscine accessi e conservazione. [F13]

Variabili, segreti e identità

Una variabile non è automaticamente un segreto. Regione, nome del gruppo e tag dell'immagine sono configurazione. Password e token sono credenziali. Identificatori come tenant ID e client ID non sono password, ma possono comunque essere gestiti come configurazione riservata secondo le regole organizzative.

Quando una pipeline accede ad Azure, preferisci un'identità federata con token temporanei rispetto a una password applicativa di lunga durata. La federazione deve limitare repository, contesto e ambiente autorizzati: non basta attivare una singola opzione nel file YAML. [F14]

Il controllo prima dell'apply

Leggi l'elenco delle risorse da creare, cambiare o distruggere. Una sostituzione può interrompere un servizio o eliminare dati. Se il piano non corrisponde all'intenzione, fermati e correggi la configurazione. Non usare l'approvazione automatica come rimedio a un processo poco compreso.

Esercizio guidato: dopo il primo apply, ripeti `terraform plan`. Se non hai modificato configurazione o ambiente, l'esito atteso è l'assenza di cambiamenti. Cambia poi un tag e identifica la differenza. Documenta perché la modifica proposta è coerente con l'obiettivo.

Dismissione

Per il gruppo Terraform usa `terraform plan -destroy` per esaminare ciò che verrebbe eliminato e, quando previsto, `terraform destroy`, leggendo la conferma. Prima verifica che nessuno abbia inserito risorse estranee nel gruppo. Non eliminare lo state per «rimuovere» l'infrastruttura: le risorse Azure resterebbero attive.

4.5 • CI/CD e lettura di una pipeline YAML

La Continuous Integration verifica frequentemente le modifiche integrate. La Continuous Delivery prepara versioni distribuibili con un passaggio di rilascio controllato; la Continuous Deployment automatizza anche quel passaggio quando i controlli lo consentono. Le organizzazioni possono adottare combinazioni differenti.

Una pipeline è una procedura eseguibile. Il file YAML ne descrive eventi, job e passi; l'indentazione è significativa. Un errore di spaziatura può cambiare la struttura o impedire l'avvio. Usa un editor con controllo YAML e leggi i log del singolo passo che fallisce.

Parti di un workflow

Elemento	Funzione	Esempio nel progetto
Trigger	Stabilisce quando partire	Pull request, push o avvio manuale
Runner/agent	Esegue i comandi	Macchina Linux con Docker
Job	Raggruppa passi e permessi	Verifica dell'applicazione
Step	Esegue un comando o un'azione	Build e richiesta HTTP
Artifact	Conserva un risultato	Report oppure immagine nel registry
Environment	Identifica una destinazione e le sue regole	Ambiente didattico

Verificare una build

La build produce un'immagine, ma il test deve dimostrare almeno un comportamento. Nel progetto controlliamo la presenza del titolo e la risposta della pagina health. Sono prove essenziali, non una copertura completa di funzionalità o sicurezza.

Promuovere lo stesso artifact tra ambienti riduce differenze involontarie. Ricostruire l'immagine a ogni passaggio può introdurre dipendenze cambiate anche quando il sorgente è uguale. Associa sempre commit, immagine e deployment.

Errori e rollback

Se un test fallisce, non eseguire il deployment. Se il deployment termina ma il servizio non risponde, la pipeline non deve essere considerata riuscita. Una strategia di rollback richiede una versione precedente nota e ancora disponibile; non può essere improvvisata cercando «l'ultimo tag» dopo il guasto.

Ripasso: che differenza c'è tra build riuscita e rilascio riuscito? Perché un tag `latest` non basta a identificare una versione? Cosa deve essere controllato prima di applicare un piano infrastrutturale?

Laboratorio 4A • Verifiche automatiche in GitHub

Salva il workflow in `.github/workflows/ci.yml`. Questo esempio non accede ad Azure e può essere usato sulle pull request. Richiede che i file dell'applicazione e del container siano nella radice e nelle cartelle previste.

```
name: Verifica Academy Portal
on:
  pull_request:
  push:
    branches: [main]
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Costruisce l'immagine
        run: docker build -t academy:test .
      - name: Avvia e verifica
        shell: bash
        run: |
          set -euo pipefail
          docker run -d --name academy-test \
            -p 127.0.0.1:8080:8080 academy:test
          for i in {1..20}; do
            if curl -fsS http://127.0.0.1:8080/health.html \
              -o /tmp/health; then
              break
            fi
            sleep 1
          done
          grep -qx 'ok' /tmp/health
          curl -fsS http://127.0.0.1:8080/ > /tmp/home
          grep -q 'Academy Portal' /tmp/home
      - name: Log e pulizia
        if: always()
        run: |
          docker logs academy-test || true
          docker rm -f academy-test || true
```

Prova positiva e negativa

Apri una pull request e osserva i passi. Poi modifica temporaneamente il titolo eliminando la stringa attesa: la verifica deve fallire. Ripristina il contenuto e controlla il nuovo esito. Configura con il docente il controllo obbligatorio prima del merge.

Le azioni sono indicate tramite tag major per leggibilità didattica; in un contesto controllato fissale a SHA verificati e mantienile aggiornate. Non eseguire codice non revisionato con credenziali cloud. **Consegna:** URL o identificativo delle due esecuzioni e spiegazione del test fallito.

Laboratorio 4B • Deployment infrastrutturale

Questo workflow separato distribuisce la rete Bicep e viene avviato manualmente. Salvalo in `.github/workflows/infra.yml`. Non distribuisce l'applicazione: quel passaggio viene completato dopo la preparazione di AKS.

Configurazione preliminare del docente

Predisporre un'identità Azure con ruolo sufficiente sul gruppo didattico e credenziale federata per il repository e l'environment `lab`. Configurare in GitHub i valori `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID` come secrets e `AZURE_RESOURCE_GROUP` come variabile. Impostare regole dell'environment per il branch `main` e revisione, se supportata dal piano usato.

Il subject della federazione deve corrispondere al formato effettivo dei token del repository; non copiare alla cieca un subject da un esempio precedente. `id-token: write` abilita la richiesta del token OIDC, non concede ruoli Azure. [F14]

```
name: Infrastruttura del laboratorio
on: workflow_dispatch
permissions:
  contents: read
  id-token: write
jobs:
  deploy:
    if: github.ref == 'refs/heads/main'
    environment: lab
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
      - name: Anteprima e deployment
        env:
          LAB_RG: ${{ vars.AZURE_RESOURCE_GROUP }}
        run: |
          az deployment group what-if -g "$LAB_RG" \
            --template-file infra/main.bicep
          az deployment group create -g "$LAB_RG" \
            --template-file infra/main.bicep
```

Limite didattico: in questo esempio il what-if e l'apply sono consecutivi. In un flusso con approvazione del piano, separa anteprima e applicazione con un controllo esplicito sul risultato dell'anteprima. L'approvazione dell'environment prima del job non equivale ad approvare un piano ancora da calcolare.

4.6 • La stessa logica in Azure Pipelines

In Azure DevOps il repository può essere ospitato in Azure Repos; le revisioni avvengono tramite pull request e le automazioni tramite Azure Pipelines. Il concetto centrale resta invariato: un evento avvia un agent che esegue passi descritti in un file versionato.

Esempio di pipeline di build

Questo file `azure-pipelines.yml` verifica la costruzione dell'immagine. È un esempio alternativo alla build GitHub, non un passaggio necessario del progetto. L'organizzazione deve disporre di un agent e dei permessi per eseguirlo.

```
trigger:
  branches:
    include:
      - main
pool:
  vmImage: ubuntu-latest
steps:
  - checkout: self
  - script: |
      docker build -t academy:$(Build.BuildId) .
    displayName: Build del container
  - script: |
      docker run --rm academy:$(Build.BuildId) -t
    displayName: Verifica configurazione NGINX
```

Il secondo passo passa `-t` a NGINX, attraverso l'ENTRYPOINT del Dockerfile, per verificare la configurazione. Non sostituisce il test HTTP del laboratorio 4A. Aggiungi una prova applicativa prima di considerare completa la pipeline.

Collegamento ad Azure

Per distribuire su Azure, configura una service connection con identità e scope controllati, preferibilmente tramite federazione quando supportata. Non inserire password nel file YAML. Autorizza soltanto le pipeline che devono usare la connessione.

Le policy di validazione delle pull request in Azure Repos si configurano sulle regole del branch; non presumere che qualunque configurazione `pr` in YAML produca automaticamente la stessa protezione.

Laboratorio comparativo

Individua, nei due sistemi, dove si impostano trigger, agent, variabili, permessi e protezione degli ambienti. Scrivi una breve tabella di equivalenza. Non occorre mantenere due pipeline operative per la stessa applicazione: lo scopo è riconoscere i concetti trasferibili.

Evidenza di fine modulo: file IaC, anteprima commentata, workflow di test con prova negativa e deployment infrastrutturale documentato. Le credenziali e lo state restano fuori dal repository.

5.1 • Kubernetes: stato desiderato e riconciliazione

Kubernetes coordina l'esecuzione di applicazioni containerizzate. Il discente descrive uno stato desiderato, per esempio due repliche di un'applicazione; i controller confrontano continuamente questo stato con quello osservato e tentano di ridurre le differenze.

L'orchestratore non comprende da solo il significato aziendale dell'applicazione. Se chiedi due repliche di un'immagine sbagliata, può distribuire correttamente proprio quell'immagine. Servono manifest corretti, controlli di salute e verifica funzionale.

Control plane e worker node

Il control plane espone l'API e coordina il cluster. L'API server riceve le richieste; lo scheduler sceglie dove collocare i Pod; i controller gestiscono la riconciliazione; il datastore conserva lo stato del cluster. I worker node eseguono i container tramite un runtime e il kubelet, che interagisce con il control plane.

Un **Pod** è l'unità minima distribuibile: può contenere uno o più container che condividono rete e risorse previste dal Pod. Per la nostra applicazione ogni Pod contiene un web server. Un **Deployment** gestisce repliche e aggiornamenti dei Pod attraverso ReplicaSet. [F15]

Namespace e label

I namespace organizzano oggetti e scope di molte operazioni. Non sono, da soli, un confine completo di sicurezza o isolamento di rete. Le label identificano gruppi di oggetti; i selector permettono ad altri componenti di individuarli.

Una label errata può lasciare un Service senza backend anche quando i Pod funzionano. Per questo i nomi delle risorse, le label e i selector vanno letti separatamente: il Service non seleziona un Deployment per il semplice fatto che hanno lo stesso nome.

Leggere YAML Kubernetes

Un manifest contiene `apiVersion`, `kind`, `metadata` e, per molte risorse, `spec`. `metadata` descrive identità ed etichette; `spec` contiene lo stato desiderato. `status` è tipicamente popolato dal sistema e non deve essere copiato indiscriminatamente in un manifest sorgente.

Esercizio: se cancelli un Pod controllato da un Deployment con due repliche, che cosa dovrebbe succedere? Perché cancellare ripetutamente il Pod non corregge un errore permanente nella sua configurazione?

5.2 • Deployment, risorse e health check

Un Deployment mantiene un numero desiderato di repliche e gestisce il passaggio da una versione all'altra. Un aggiornamento del template dei Pod genera una nuova revisione. Il numero di repliche disponibili può variare durante il rollout secondo la strategia impostata. [F15]

Requests e limits

Le **requests** dichiarano le risorse considerate per il posizionamento. I **limits** impongono limiti d'uso secondo le caratteristiche della risorsa. La CPU può subire throttling; un container che supera il limite di memoria può essere terminato per memoria insufficiente. Richieste eccessive possono lasciare i Pod in Pending anche se i nodi sembrano poco utilizzati.

Per Academy Portal iniziamo con valori didattici piccoli e li misuriamo. Non sono un dimensionamento universale: contenuti, traffico e configurazione possono richiedere risorse diverse.

Tre tipi di probe

Probe	Domanda	Conseguenza tipica del fallimento
Readiness	Il container può ricevere traffico?	Il Pod non è pronto come backend del Service
Liveness	Il processo è in una condizione da cui non recupera?	Kubernetes può riavviare il container
Startup	L'avvio è terminato?	Ritarda le altre probe finché l'avvio non è riuscito

Una liveness troppo aggressiva può causare riavvii continui. Non usarla per «aggiustare» un'applicazione lenta senza capire la causa. Una readiness su una pagina statica prova soltanto la capacità di servirla; un'applicazione con database richiede una progettazione più attenta dei controlli.

Scalabilità e disponibilità

Aumentare le repliche distribuisce il carico se l'applicazione e le sue dipendenze lo consentono. L'Horizontal Pod Autoscaler regola le repliche usando metriche e criteri; il cluster autoscaler riguarda la capacità dei nodi. Sono livelli distinti.

Due repliche sullo stesso nodo non garantiscono continuità in caso di guasto del nodo. Un laboratorio con un solo nodo insegna i meccanismi, ma non rappresenta un'architettura di produzione ad alta disponibilità.

Esercizio: descrivi tre cause diverse di Pod Pending e un controllo utile per ciascuna. Poi spiega perché un container Running può appartenere a un Pod non Ready.

5.3 • Service, Ingress e accesso all'applicazione

Gli indirizzi dei Pod possono cambiare. Un Service fornisce un punto di accesso stabile e seleziona i Pod tramite label. La porta del Service e quella del container possono differire: nel nostro esempio il Service usa 80 e inoltra a 8080. [F16]

Tipi di accesso

`ClusterIP` espone il servizio all'interno del cluster. `NodePort` utilizza una porta sui nodi. `LoadBalancer` richiede l'integrazione infrastrutturale per offrire un bilanciatore; nel cloud può creare risorse a pagamento. Per il laboratorio useremo ClusterIP e port-forward locale, così da verificare il servizio senza pubblicarlo direttamente.

Ingress e controller

Un Ingress descrive regole HTTP/HTTPS verso Service, ad esempio per hostname e percorso. Il solo oggetto Ingress non instrada traffico: serve un controller compatibile installato e configurato. L'IngressClass deve corrispondere al controller scelto. La scelta e la manutenzione del controller spettano al contesto del cluster. [F17]

Esempio da adattare a un controller predisposto dal docente; `academy.example.com` è un dominio illustrativo, non un indirizzo già disponibile.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: academy
  namespace: academy
spec:
  ingressClassName: <CLASSE_INSTALLATA>
  rules:
    - host: academy.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: academy-web
                port:
                  number: 80
```

Per HTTPS occorrono configurazione TLS, certificato valido e DNS coerente. Non pubblicare questo esempio così com'è: manca la parte TLS e il dominio deve essere realmente gestito. L'API Ingress resta un concetto del corso; Gateway API è un possibile approfondimento per nuovi progetti, da valutare con la documentazione del controller. [F17]

Diagnosi: se un Service non ha endpoint pronti, controlla selector e readiness prima di modificare l'Ingress.

5.4 • ConfigMap, Secret e dati persistenti

Separare immagine e configurazione

Un'immagine dovrebbe poter essere usata in più ambienti con configurazioni esterne. Una ConfigMap contiene valori non riservati; un Secret è destinato a informazioni sensibili. Entrambi possono essere utilizzati come variabili d'ambiente o file montati, secondo il caso.

Cambiare una ConfigMap non aggiorna automaticamente le variabili d'ambiente di un processo già avviato. Per valori montati come file, l'aggiornamento e la rilettura dipendono dalla modalità di montaggio e dal comportamento dell'applicazione. Definisci una procedura di rollout esplicita quando serve.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: academy-config
  namespace: academy
data:
  APP_ENV: "lab"
```

Per provarla aggiungi al container del Deployment una sezione `envFrom` che referenzi `configMapRef` con `name: academy-config`. La nostra pagina statica non legge questa variabile: verifica l'iniezione con un comando nel container, senza aspettarti che il testo HTML cambi da solo.

Segreti e identità del workload

La codifica base64 di un Secret non è cifratura. Chi può leggere il Secret può recuperare il valore. Occorre limitare gli accessi e valutare la protezione dei dati a riposo. Non salvare credenziali reali nei manifest del repository, neppure codificate. [F18]

In AKS, Workload Identity permette a un Pod correttamente configurato di accedere a servizi Azure tramite un'identità federata. L'identità del workload, quella del cluster e quella usata dai nodi per scaricare immagini non sono intercambiabili. Un errore di pull dal registry non si corregge assegnando casualmente permessi all'applicazione. [F19]

Persistenza e microservizi

Un volume persistente separa la durata dei dati da quella del Pod. Una PersistentVolumeClaim richiede capacità con determinate caratteristiche. Non basta aumentare le repliche di un database per ottenere una replica dei dati corretta.

Un sistema a microservizi divide responsabilità applicative in servizi comunicanti. Il vantaggio di distribuzioni indipendenti porta anche complessità di rete, osservabilità e gestione dei guasti. Il portale statico del corso è un singolo servizio: l'estensione a due servizi è proposta nel progetto finale, senza introdurre database non necessari.

5.5 • AKS: preparare il laboratorio

Azure Kubernetes Service gestisce componenti del control plane; il team mantiene responsabilità su workload, accessi, configurazioni e scelte operative. Prima del cluster definisci rete, capacità, identità, registry e modalità di accesso all'API.

Prerequisiti verificati dal docente

Occorrono quote per lo SKU scelto, provider Azure richiesti registrati, permessi per creare il cluster e assegnare ruoli all'identità di pull. Il registry del modulo 3 deve essere raggiungibile. La modalità RBAC del registry è il presupposto dell'esempio `--attach-acr`; con ACR ABAC occorrono ruoli e procedura differenti. [F09]

Il cluster può essere condiviso dalla classe usando namespace distinti e permessi separati. In tal caso ciascun discente sostituisce il namespace `academy` con quello assegnato in tutti i file e comandi. Non creare un cluster per discente senza una scelta consapevole su budget e quote.

Esempio di creazione guidata

Questo blocco è destinato al docente o a un account esplicitamente autorizzato. Usa un nodo per finalità didattiche; lo SKU è un esempio, non una garanzia di disponibilità o costo minimo.

```
LAB_AKS='aks-academy'
LAB_ACR_ID=$(az acr show -n "$LAB_ACR" --query id -o tsv)
az aks create -g "$LAB_RG" -n "$LAB_AKS" \
  --node-count 1 --node-vm-size Standard_D2s_v5 \
  --enable-managed-identity --generate-ssh-keys \
  --attach-acr "$LAB_ACR_ID"
az aks get-credentials -g "$LAB_RG" -n "$LAB_AKS"
kubectl config current-context
kubectl get nodes
kubectl create namespace academy
```

Non usare `--admin` come soluzione predefinita a un errore di autorizzazione. In cluster integrati con Microsoft Entra il docente deve predisporre login, kubelogin e autorizzazioni Kubernetes/Azure appropriate. Il cluster dell'esempio e quello aziendale possono avere autenticazioni differenti.

Esito atteso: contesto corretto e nodo Ready. Un fallimento di creazione richiede lettura del dettaglio di quota, rete, policy o autorizzazione. Evita tentativi ripetuti con nomi diversi: possono lasciare risorse parziali e costi.

Laboratorio 5A • Il Deployment del portale

Prerequisiti: cluster accessibile, namespace creato e immagine del modulo 3 presente in ACR per l'architettura dei nodi. L'identità di pull del cluster deve essere autorizzata. Salva in `k8s/deployment.yaml` e sostituisci `<LOGIN_SERVER>` e `<TAG>` con i valori effettivi.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: academy-web
  namespace: academy
spec:
  replicas: 2
  selector:
    matchLabels:
      app: academy-web
  template:
    metadata:
      labels:
        app: academy-web
    spec:
      containers:
        - name: web
          image: <LOGIN_SERVER>/academy-portal:<TAG>
          ports:
            - containerPort: 8080
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
          resources:
            requests:
              cpu: "100m"
              memory: "64Mi"
            limits:
              cpu: "500m"
              memory: "128Mi"
          readinessProbe:
            httpGet:
              path: /health.html
              port: 8080
            initialDelaySeconds: 2
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /health.html
              port: 8080
            initialDelaySeconds: 10
            periodSeconds: 10
```

Le label del template e il selector devono corrispondere. L'immagine viene eseguita con l'utente definito nel Dockerfile. Le impostazioni mostrate sono una base didattica, non un insieme completo di controlli per la produzione.

Laboratorio 5B • Service e verifica funzionale

Salva questo secondo manifest in `k8s/service.yaml`.

```
apiVersion: v1
kind: Service
metadata:
  name: academy-web
  namespace: academy
spec:
  type: ClusterIP
  selector:
    app: academy-web
  ports:
    - port: 80
      targetPort: 8080
```

Applicare e osservare

```
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl -n academy rollout status deployment/academy-web \
  --timeout=180s
kubectl -n academy get pods -o wide
kubectl -n academy get service academy-web
kubectl -n academy get endpointslices \
  -l kubernetes.io/service-name=academy-web
kubectl -n academy port-forward service/academy-web 8080:80
```

Il port-forward resta attivo nel terminale. Apri un secondo terminale ed esegui `curl --fail http://127.0.0.1:8080/` e la richiesta a `/health.html`. Al termine interrompi il port-forward con Ctrl+C.

Esito atteso: due repliche pronte, backend individuati dal Service, titolo corretto e health `ok`. Il port-forward è una verifica locale, non un endpoint pubblico persistente e non prova il funzionamento di un eventuale Ingress.

Aggiornare e recuperare

Cambia il testo della home, crea un commit e pubblica una nuova immagine con nuovo tag. Aggiorna il manifest e applicalo. Controlla `rollout status`, l'immagine nei Pod e il testo restituito. Conserva il tag precedente.

Per provare un recupero controllato, usa `kubectl -n academy rollout history deployment/academy-web`; se esiste una revisione valida precedente, `kubectl -n academy rollout undo deployment/academy-web` ne ripristina il template. Dopo il rollback riallinea anche il manifest nel repository: altrimenti il prossimo apply può reintrodurre la versione sbagliata.

Consegna: manifest, immagini con due tag, verifica prima/dopo e nota sul rollback. Il rollback del Deployment non ripristina automaticamente dati esterni o tutte le ConfigMap.

5.6 • Helm: pacchetti e configurazione

Helm organizza manifest Kubernetes in chart parametrizzabili. Una chart è il pacchetto; una release è un'installazione della chart con valori specifici. La stessa chart può generare risorse per ambienti diversi. Prima di installare esamina sempre ciò che verrà prodotto. [F20]

Esempio minimo separato

Per evitare conflitti con `academy-web`, crea una chart che generi un Deployment con un nome derivato dalla release. In `charts/academy/Chart.yaml` inserisci:

```
apiVersion: v2
name: academy
version: 0.1.0
appVersion: "1.0"
```

In `charts/academy/values.yaml` inserisci `replicaCount: 1` e la proprietà `image` con la stringa completa dell'immagine ACR. In `charts/academy/templates/deployment.yaml` salva:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-web
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Release.Name }}-web
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}-web
    spec:
      containers:
        - name: web
          image: {{ .Values.image | quote }}
          ports:
            - containerPort: 8080
```

Questa chart minima serve a comprendere il rendering: manca di Service, probe e requests/limits. Completarla è parte dell'esercizio, riportando i campi già studiati. Non usare due strumenti per gestire lo stesso Deployment senza una migrazione esplicita.

```
helm lint charts/academy
helm template academy-helm charts/academy -n academy
helm upgrade --install academy-helm charts/academy \
  -n academy --wait --timeout 3m
helm list -n academy
```

Per verificare la release usa `helm status`; per esaminare revisioni usa `helm history`; un rollback richiede il numero di una revisione esistente. `helm uninstall academy-helm -n academy` rimuove la release didattica. Verifica separatamente eventuali risorse persistenti.

5.7 • Troubleshooting Kubernetes e AKS

La diagnosi deve restringere il problema, non accumulare tentativi. Parti dall'oggetto che mostra il sintomo e ricostruisci dipendenze e configurazione. Controlla sempre cluster e namespace prima di modificare risorse.

Sintomo	Cause da verificare	Prima evidenza utile
Pending	Risorse insufficienti, vincoli, volume non disponibile	Events e describe del Pod
ImagePullBackOff	Nome/tag errato, permessi ACR, rete	Messaggio del pull negli Events
CrashLoopBackOff	Processo termina, configurazione o dipendenze errate	Log correnti e precedenti
Running, non Ready	Readiness fallita, porta o percorso errato	Probe, eventi e risposta HTTP
Service senza backend	Selector non corrispondente, Pod non Ready	Label ed EndpointSlice
OOMKilled	Memoria superata	Stato terminato, limiti e misure

Comandi da leggere prima di agire

Sostituisci il nome del Pod con quello realmente restituito dal cluster.

```
kubectl config current-context
kubectl -n academy get pods
kubectl -n academy describe pod '<NOME_POD>'
kubectl -n academy logs '<NOME_POD>' -c web --tail=100
kubectl -n academy logs '<NOME_POD>' -c web --previous
kubectl -n academy get events --sort-by=.lastTimestamp
kubectl -n academy get pods --show-labels
```

`--previous` è utile se esiste un'istanza precedente del container terminata. L'assenza di log precedenti non è, da sola, un nuovo guasto. `kubectl top pods` richiede la disponibilità del servizio di metriche; se manca, non significa che i Pod consumino zero risorse.

Un incidente spiegato

Il portale non risponde, ma i Pod sono Running. Gli EndpointSlice non riportano backend pronti. Controllando la readiness emerge un 404: il percorso della probe è `/ready`, mentre nell'immagine esiste `/health.html`. La correzione riguarda il manifest; ampliare gli accessi di rete non risolve questa causa.

Documenta sintomo, ora, versione, ipotesi, evidenza, correzione e verifica. Il report deve permettere a un collega di capire perché la soluzione era appropriata.

Ripasso: perché un Secret in base64 non è protetto? Cosa manca a un Ingress perché funzioni? Qual è la differenza tra aumentare repliche e aumentare nodi?

5.8 • Automatizzare il rilascio applicativo

Il collegamento finale tra CI/CD e AKS consiste nel distribuire una versione identificabile e verificarla. Per non introdurre credenziali nel repository, il runner deve essere predisposto dal docente con strumenti, federazione e autorizzazioni corrette.

Presupposti del flusso

Usa un agent autorizzato con Azure CLI, Docker, kubectl e, per cluster Entra, kubelogin. L'identità deve poter pubblicare nel registry, recuperare la configurazione del cluster e aggiornare risorse nel namespace. L'accesso all'API deve essere consentito dalla rete del runner. Avere Contributor sul gruppo non dimostra automaticamente di poter aggiornare un Deployment Kubernetes.

Se il cluster usa autenticazione Entra, dopo il login Azure e il recupero del kubeconfig può essere necessario `kubelogin convert-kubeconfig -l azurecli`. Non applicare questo passaggio a configurazioni diverse senza capire il meccanismo usato.

Script di deployment dell'applicazione

Salva `scripts/deploy.sh` e invocalo con `bash scripts/deploy.sh` dal job autenticato, soltanto dopo build e test. Definisci `LAB_RG`, `LAB_AKS`, `LAB_LOGIN_SERVER` e `LAB_TAG` come variabili del job. I manifest del laboratorio devono essere già stati applicati; lo script aggiorna il Deployment esistente.

```
set -euo pipefail
: "${LAB_RG:?Variabile mancante}"
: "${LAB_AKS:?Variabile mancante}"
: "${LAB_LOGIN_SERVER:?Variabile mancante}"
: "${LAB_TAG:?Variabile mancante}"
az aks get-credentials -g "$LAB_RG" -n "$LAB_AKS" \
  --overwrite-existing
# Solo per il cluster Entra predisposto:
# kubelogin convert-kubeconfig -l azurecli
kubectl -n academy set image deployment/academy-web \
  "web=$LAB_LOGIN_SERVER/academy-portal:$LAB_TAG"
kubectl -n academy rollout status deployment/academy-web \
  --timeout=180s
kubectl -n academy get deployment academy-web \
  -o jsonpath='{.spec.template.spec.containers[0].image}'
```

Aggiungi al job una verifica HTTP attraverso il percorso autorizzato e confronta il contenuto atteso. Non dichiarare il rilascio concluso solo perché `set image` è riuscito. Questo aggiornamento è imperativo: registra il nuovo riferimento immagine anche nel manifest o nei valori Helm, così che la configurazione versionata resti coerente.

Progetto finale • Dal repository al cluster

Scenario e obiettivo

Un ente di formazione vuole pubblicare un portale informativo del corso. Il team deve poter modificarne i contenuti, verificare automaticamente la build e distribuire una versione riconoscibile nel laboratorio Azure. La soluzione deve essere documentata in modo che un altro discente possa riprodurla.

Non sono richiesti dati personali reali, un database o un dominio pubblico. Il risultato minimo è un'applicazione statica raggiungibile tramite il percorso di laboratorio, con immagine in ACR, manifest versionati e prove di verifica.

Consegne richieste

Area	Evidenza verificabile
Progettazione	Schema delle risorse e motivazione delle scelte
Azure	Gruppo, rete, identità e controllo dei costi documentati
Git	Storia leggibile e almeno una pull request revisionata
Container	Dockerfile, configurazione, tag e digest dell'immagine
IaC	Bicep o Terraform con anteprima spiegata
CI/CD	Test positivo e negativo; procedura di rilascio tracciabile
Kubernetes	Deployment, Service e prova di aggiornamento/rollback
Operatività	Report di un guasto simulato e checklist di pulizia

Estensione a due servizi

Aggiungi un secondo servizio statico, per esempio `academy-catalogo`, con una propria immagine e Deployment. Esponilo con un Service distinto e dimostra che può essere aggiornato senza ricostruire il portale. Se il docente ha predisposto un controller, configura percorsi distinti tramite Ingress; altrimenti verifica i servizi separatamente.

Spiega cosa cambierebbe se il portale dovesse chiamare un'API con dati persistenti: gestione dei timeout, errori delle dipendenze, autenticazione, contratti e osservabilità. Non presentare due container come microservizi autonomi senza identificare le loro responsabilità.

Criteri formativi proposti

Attribuisci 0-2 punti a ciascuna delle cinque aree: correttezza funzionale, riproducibilità, gestione degli accessi, diagnosi, chiarezza della documentazione. 0 significa evidenza assente, 1 risultato parziale o poco spiegato, 2 risultato corretto e motivato. La griglia è un supporto didattico, non una regola ufficiale di valutazione del corso.

Discussione finale: illustra una scelta che semplifica la soluzione, un limite del laboratorio rispetto alla produzione e un errore che sapresti ora diagnosticare senza procedere per tentativi casuali.

Checklist operativa e chiusura del laboratorio

Prima di lavorare

Verifica sottoscrizione, gruppo e contesto Kubernetes. Leggi l'operazione proposta, identifica le risorse coinvolte e controlla se la modifica può interrompere un servizio o eliminare dati. Se usi la pipeline, verifica branch, environment e artifact selezionato.

Dopo ogni modifica

Confronta stato atteso e osservato; controlla risposta applicativa, log e risorse create. Aggiorna repository e documentazione. Non lasciare differenze manuali senza spiegazione, perché rendono poco affidabile il prossimo deployment.

Ordine di dismissione

1. Conserva file, commit, evidenze e configurazioni necessarie; escludi segreti.

2. Rimuovi la release Helm di esercizio, se creata, e verifica gli oggetti residui.

3. Se il cluster è condiviso, elimina soltanto le risorse o il namespace assegnati, secondo le indicazioni del docente. Non eliminare il cluster comune.

4. Per risorse gestite da Terraform, esamina il piano di distruzione e usa Terraform per rimuoverle, mantenendo lo state fino alla conclusione.

5. Se il gruppo Azure è interamente personale e destinato alla dismissione, verifica tutto il contenuto prima di eliminarlo.

6. Controlla anche risorse esterne al gruppo: nodi gestiti AKS, volumi, registry separati, log e componenti di rete. Attendi l'aggiornamento dei dati di costo.

Comandi di controllo ed eliminazione

Il comando di eliminazione richiede conferma: non aggiungere opzioni per saltarla in un'esercitazione senza aver verificato il contenuto.

```
az resource list -g "$LAB_RG" -o table
# Solo per il gruppo didattico interamente da eliminare:
az group delete --name "$LAB_RG"
az group exists --name "$LAB_RG"
```

L'esito `false` conferma l'assenza di quel gruppo, non l'assenza di qualunque costo nella sottoscrizione. Un resource group separato gestito da Terraform non viene eliminato da questo comando.

Scheda di autovalutazione

Per ogni modulo indica: «riesco in autonomia», «riesco con guida» oppure «da ripassare». Aggiungi un'evidenza concreta: un commit, un test, un output interpretato o un incidente risolto. Scrivere «conosco Kubernetes» è meno utile di «so capire perché un Service non trova Pod pronti».

Ripasso commentato • Cloud, Azure e Git

1. Quando scegliere PaaS invece di IaaS?

Quando i requisiti applicativi sono compatibili con la piattaforma gestita e non serve controllare il sistema operativo. Per un portale standard, ridurre manutenzione e patch della VM può essere vantaggioso. La risposta deve comunque considerare rete, runtime, costi e vincoli dell'applicazione.

2. Resource group e subnet sono equivalenti?

No. Il gruppo organizza risorse e il loro ciclo di vita; la subnet suddivide lo spazio di una rete. Risorse nello stesso gruppo non sono automaticamente nella stessa rete e non ottengono accesso reciproco per quel solo motivo.

3. Un budget impedisce di superare la soglia?

No. È uno strumento di controllo e notifica; non è, da solo, un limite di spesa bloccante. Per contenere i costi occorrono scelte operative su durata, capacità, servizi e dismissione. I dati di costo e gli avvisi possono arrivare con ritardo. [F03]

4. Perché backup e ridondanza non coincidono?

La ridondanza mantiene copie per resistere a guasti previsti dal servizio; un errore logico può propagarsi alle copie. Il backup, con conservazione e ripristino verificati, permette di recuperare uno stato precedente. La protezione concreta dipende dal servizio e dalla configurazione adottata.

5. Cosa cambia tra autenticazione e autorizzazione?

La prima dimostra l'identità; la seconda valuta se quell'identità può compiere l'operazione. Un account correttamente autenticato può non avere il ruolo dati necessario. La diagnosi deve identificare operazione, scope e identità effettiva.

6. Una VM deallocata non costa più nulla?

Non necessariamente. Il calcolo può smettere di essere fatturato mentre dischi, indirizzi o altri servizi restano a pagamento. La verifica va fatta sulle singole risorse e sui dati di costo, non soltanto sullo stato della VM.

7. Commit, push e pull request sono la stessa cosa?

No. Il commit registra una modifica nella storia locale; il push trasferisce riferimenti e oggetti al remoto; la pull request propone un'integrazione e organizza la revisione sulla piattaforma. Una modifica non è condivisa solo perché è stata salvata nell'editor.

8. Basta aggiungere un segreto a .gitignore?

No. Se il segreto è già stato tracciato, può essere presente nella storia. Va revocato o ruotato e occorre gestire la rimozione con una procedura coordinata. `.gitignore` serve a prevenire inclusioni future, non a cancellare retroattivamente i dati.

Ripasso commentato • Automazione e Kubernetes

9. Build riuscita significa applicazione funzionante?

No. Una build dimostra che l'immagine è stata prodotta. Servono avvio, test e verifica del servizio distribuito. Anche una pipeline verde può essere incompleta se non controlla il comportamento che interessa agli utenti.

10. Perché evitare latest come unico riferimento?

Perché un tag può puntare nel tempo a contenuti diversi. Un tag legato al commit e il digest consentono di identificare ciò che è stato distribuito. Per un rollback affidabile occorre una versione precedente effettivamente disponibile.

11. Che cosa va letto in un piano IaC?

Le risorse create, modificate, sostituite o distrutte, le dipendenze e gli effetti sui dati. Un nome cambiato può causare una nuova risorsa. Un piano sintatticamente valido può essere operativamente sbagliato: va confrontato con l'intenzione del cambiamento.

12. Cosa succede cancellando un Pod di un Deployment?

Se il Deployment richiede ancora quella capacità, il controller tenta di creare un Pod sostitutivo. La sostituzione non corregge una configurazione errata o un'immagine non disponibile: il nuovo Pod può presentare lo stesso problema.

13. Un Pod Running è necessariamente Ready?

No. Il processo può essere attivo ma la readiness può fallire. Potrebbe esserci un percorso errato, un avvio incompleto o una dipendenza non disponibile. Controlla eventi, probe e risposta dell'applicazione prima di modificare la rete.

14. Secret e Ingress: quali equivoci evitare?

Un Secret in base64 è decodificabile; la sicurezza dipende da accessi e protezione della piattaforma. Un Ingress senza controller non produce automaticamente instradamento. Servono inoltre Service, backend pronti e, per un hostname reale, DNS coerente. [F17, F18]

15. Scalare Pod o nodi: cosa cambia?

Le repliche aumentano le istanze applicative. I nodi aumentano la capacità del cluster. Se mancano CPU o memoria per schedulare, aggiungere repliche può generare Pod Pending. Se il collo di bottiglia è esterno, più nodi potrebbero non migliorare la risposta.

16. Il rollback annulla ogni cambiamento?

No. Il rollback di un Deployment riguarda il template gestito e le revisioni disponibili. Non ripristina automaticamente database, file esterni o tutte le configurazioni. Verifica il servizio e riallinea il repository per evitare che il successivo rilascio riproponga l'errore.

Glossario • Cloud, sviluppo e automazione

Termine	Significato operativo
API	Interfaccia con cui un software richiede operazioni a un altro sistema
Artifact	Risultato identificabile di un processo, come immagine o report
Azure CLI	Strumento da riga di comando per gestire Azure
Bicep	Linguaggio dichiarativo per descrivere risorse Azure
Branch	Riferimento a una linea di sviluppo in Git
CI/CD	Pratiche di integrazione, verifica e distribuzione continua
Commit	Registrazione coerente di una modifica nella storia Git
Container	Istanza isolata di processi avviata da un'immagine
Digest	Identificatore derivato dal contenuto di un'immagine
Drift	Divergenza tra configurazione attesa e ambiente effettivo
Environment	Ambiente di destinazione con regole e configurazioni proprie
IaC	Gestione dell'infrastruttura attraverso file versionabili
Identità gestita	Identità Azure utilizzabile senza gestire direttamente un segreto
Immagine	Pacchetto usato per avviare container
OIDC	Protocollo usato anche per federare identità con token temporanei
Pipeline	Sequenza automatizzata di attività e verifiche
Pull request	Proposta di integrazione con discussione e revisione
RBAC	Controllo degli accessi basato su ruoli e ambiti
Registry	Servizio che conserva e distribuisce immagini container
Resource group	Contenitore logico di risorse Azure
Runner/agent	Ambiente di esecuzione dei job di una pipeline
State	Informazioni con cui Terraform tiene traccia delle risorse gestite
Tenant	Contesto organizzativo di identità in Microsoft Entra ID
VNet	Rete virtuale privata in Azure

Usare correttamente i termini

«Deploy», «build» e «run» descrivono momenti diversi. La build produce l'immagine; il deploy aggiorna l'ambiente di destinazione; l'esecuzione avvia e mantiene i processi. Per un report chiaro specifica quale fase è fallita e quale evidenza lo dimostra.

Glossario • Kubernetes e operatività

Termine	Significato operativo
AKS	Servizio Azure per Kubernetes gestito
Chart	Pacchetto Helm che contiene template e configurazione
Cluster	Insieme di componenti di controllo e nodi di esecuzione
ConfigMap	Oggetto Kubernetes per configurazione non riservata
Control plane	Componenti che espongono l'API e coordinano il cluster
Deployment	Controller per repliche e aggiornamenti di Pod
EndpointSlice	Oggetto che descrive endpoint associabili a un Service
Helm release	Installazione di una chart con determinati valori
Ingress	Regole HTTP/HTTPS applicate da un controller compatibile
Label	Etichetta chiave-valore applicata a oggetti Kubernetes
Limit	Limite di utilizzo configurato per una risorsa del container
Liveness	Verifica usata per individuare condizioni di riavvio
Namespace	Ambito logico per molte risorse e autorizzazioni
Node	Macchina che ospita workload Kubernetes
Pod	Unità Kubernetes contenente uno o più container
Probe	Controllo di avvio, disponibilità al traffico o vitalità
Readiness	Verifica della disponibilità a ricevere traffico
ReplicaSet	Controller che mantiene un numero desiderato di Pod
Request	Risorsa dichiarata per il posizionamento del container
Rollback	Ritorno controllato a una configurazione precedente
Rollout	Introduzione progressiva di una configurazione aggiornata
Secret	Oggetto per dati sensibili, soggetto a specifici controlli
Selector	Criterio che individua oggetti attraverso le label
Service	Punto di accesso stabile a un insieme di Pod
Workload	Applicazione o attività eseguita nel cluster

Tre domande prima di intervenire

Qual è lo stato desiderato? Qual è lo stato osservato? Quale evidenza spiega la differenza? Rispondere in quest'ordine aiuta a distinguere un guasto reale, una configurazione errata e un'aspettativa non corretta.

Fonti e approfondimenti • Azure e DevOps

Programma di riferimento: «Cloud Engineer Academy - Fondamenti e competenze operative», file fornito dall'utente, 160 ore e 20 giornate. La suddivisione in cinque moduli e le relative durate sono riprese dal programma. Spiegazioni, laboratori, progetto e griglia formativa costituiscono elaborazione didattica aggiuntiva.

Documentazione tecnica: riferimenti ufficiali consultati il 9 novembre 2026. Versioni, interfacce e disponibilità dei servizi possono evolvere. Verificare i prerequisiti dell'ambiente prima delle esercitazioni. Le sigle F01-F20 rimandano ai riferimenti seguenti.

[F01] Microsoft Learn • Shared responsibility in the cloud

https://learn.microsoft.com/en-in/azure/security/fundamentals/shared-responsibility

[F02] Microsoft Learn • What is Azure role-based access control?

https://learn.microsoft.com/en-us/azure/role-based-access-control/overview

[F03] Microsoft Learn • Create and manage budgets

https://learn.microsoft.com/en-us/azure/cost-management-billing/costs/tutorial-acm-create-budgets

[F04] Microsoft Learn • What is Azure Virtual Network?

https://learn.microsoft.com/en-us/azure/virtual-network/virtual-networks-overview

[F05] Microsoft Learn • Introduction to Azure Storage

https://learn.microsoft.com/en-us/azure/storage/common/storage-introduction

[F06] Microsoft Learn • Azure Monitor overview

https://learn.microsoft.com/en-us/azure/azure-monitor/fundamentals/overview

[F07] Pro Git • Recording Changes to the Repository

https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository

[F08] Docker Docs • Building best practices

https://docs.docker.com/build/building/best-practices/

[F09] Microsoft Learn • Integrate Azure Container Registry with AKS

https://learn.microsoft.com/en-us/azure/aks/cluster-container-registry-integration

[F10] Microsoft Learn • Bicep documentation

https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/

Fonti e approfondimenti • IaC e Kubernetes

[F11] Microsoft Learn • Deploy Bicep files with the Azure CLI

https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/deploy-cli

[F12] HashiCorp Developer • Build infrastructure on Azure

https://developer.hashicorp.com/terraform/tutorials/azure-get-started/azure-build

[F13] HashiCorp Developer • Manage sensitive data in your configuration

https://developer.hashicorp.com/terraform/language/manage-sensitive-data

[F14] GitHub Docs • Configuring OpenID Connect in Azure

https://docs.github.com/en/actions/how-tos/secure-your-work/security-harden-deployments/oidc-in-azure

[F15] Kubernetes • Deployments

https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

[F16] Kubernetes • Service

https://kubernetes.io/docs/concepts/services-networking/service/

[F17] Kubernetes • Ingress

https://kubernetes.io/docs/concepts/services-networking/ingress/

[F18] Kubernetes • Secrets

https://kubernetes.io/docs/concepts/configuration/secret/

[F19] Microsoft Learn • Access and identity in AKS

https://learn.microsoft.com/en-us/azure/aks/concepts-identity

[F20] Helm • Using Helm

https://helm.sh/docs/intro/using_helm/

Proseguire lo studio

Per consolidare Azure, ripeti i laboratori cambiando un solo requisito e documentando le differenze. Per Git, lavora in coppia su una pull request con conflitto controllato. Per IaC, confronta un ambiente appena creato e uno modificato manualmente. Per Kubernetes, affronta errori diversi mantenendo la stessa applicazione, in modo da isolare il comportamento della piattaforma.

L'obiettivo non è memorizzare ogni comando. È riconoscere il problema, trovare il riferimento appropriato, prevedere gli effetti dell'operazione e verificare il risultato con evidenze ripetibili.